

δ Sandbox: Millisecond-Scale Checkpoint and Rollback for Stateful AI Agents

Paper #158

Abstract

Emerging LLM-powered AI agent workloads, such as test-time tree search and reinforcement learning, require high-frequency state exploration, relying on rapid checkpoint and rollback (C/R) of the complete sandbox state, including files and process state. Existing mechanisms duplicate the entire state, causing hundreds of milliseconds to seconds of latency per C/R, which severely bottlenecks deep search and large-scale fan-outs.

We observe that consecutive sandbox states between checkpoints differ only marginally. Therefore, instead of full duplication, a sandbox should *only duplicate the changes between consecutive checkpoints* (**Key Insight**), yet the lack of OS support makes this hard to realize. We propose a new OS-level abstraction, δ State, that delivers change-based C/R for agents through co-designed OS mechanisms. First, δ FS checkpoints file state by freezing the writable layer and inserting a new one, turning updates into copy-on-write and rollback into a layer switch. Second, δ CR quickly restores process state by forking from a frozen template process. A coordinator pairs the two so each checkpoint is one consistent (filesystem, memory) state. Building on these mechanisms, we present δ Sandbox, an agent sandbox with millisecond-level C/R. On SWE-bench, δ Sandbox hides its checkpoint work (10.83 ms) under inference and rolls back in a mean 1.86 ms, with the same warm-template fork driving low-cost RL rollout fan-out.

1 Introduction

LLM-powered agents now automate complex, multi-step tasks, from code repair [1] to web navigation [2]. An agent runs an iterative loop: it generates an action [3], executes it in a sandbox [4], observes the outcome, and refines its next step. To push capability further, agents increasingly scale test-time compute [5] through tree search such as Monte Carlo Tree Search (MCTS) [6] and Tree of Thoughts [7], exploring multiple candidate paths, checking them against execution feedback, and backtracking from failures.

In early text-based tasks (e.g., HotPotQA), backtracking cost almost nothing: the agent simply truncated its prompt history. A coding agent, by contrast, acts on real OS state [8, 9] through commands like `rm`, `pip install`, and `sed`. These side effects persist on disk and cannot be undone by editing history; reverting them requires checkpointing and restoring the complete sandbox state. Because tree search backtracks constantly, this expensive rollback lands repeatedly on the critical path, creating a severe infrastructure bottleneck.

The sandbox state encompasses two tightly coupled dimensions: durable filesystem state and ephemeral process state. These dimensions must be captured and restored jointly to prevent state divergence or context loss.

Test-time search stresses C/R along *two axes* at once: horizontally, Best-of-N sampling spreads the inference budget across many parallel trajectories, each needing a fast initial clone [5, 10]; vertically, tree search such as MCTS backtracks to arbitrary historical nodes, demanding fine-grained checkpoint/restore of intermediate states.

Beyond test-time inference, reinforcement-learning (RL) training of agent policies is a second pressure point [11–13]. Each training step must spin up k *independent* sandboxes from the same warm starting state, run them as rollouts, and tear them down. Today’s approaches rebuild this warm state per rollout with a fresh Docker container or a microVM snapshot-restore, both in the hundreds-of-milliseconds to seconds regime; with k in the tens to low hundreds, fork latency directly bounds training throughput.

Existing systems manage filesystem and process state separately, achieving C/R by duplicating the entire state into a checkpoint image [14–18]. Full duplication is acceptable for serverless cold starts [19–22], but prohibitively slow for agents that checkpoint and roll back frequently on the critical path. This creates three obstacles for agent workloads:

Challenge-1: Existing durable filesystem C/R is coarse-grained and disruptive. A significant challenge is how to efficiently checkpoint and restore the durable filesystem state of an agent sandbox, including repository edits, installed packages, caches, and temporary files. Naive file-level copying (e.g., `copytree`) scales with the directory size; Docker commits scale with the mutated layer; and VM/block snapshots [18] operate at a coarse whole-VM granularity. These approaches are tolerable for occasional resets, but too slow for per-step rollback in MCTS or large fan-out in RL. Overlayfs appears to be a natural alternative because each checkpoint could demote the current writable upper into a read-only lower and install a fresh upper, turning later writes into copy-on-write. However, stock Linux overlayfs freezes its layer stack at mount time: changing the stack requires `umount/mount`, which is impossible while an agent keeps files or working directories open and also flushes `dentry/inode` caches [23]. Moreover, ordinary overlayfs copy-up can duplicate an entire inherited file for a small edit, causing file-size-proportional write amplification.

Challenge-2: Whole-state dump and materialization limit process C/R efficiency. An agent’s process state

is not just heap bytes: it includes the Python interpreter, loaded indexes, tool context, open descriptors, and SDK helper threads that make a sandbox warm. Existing process C/R exposes a whole-state dump/materialize interface: checkpoint serializes the warm footprint, and restore reconstructs it, even when the agent changed little since the previous step. This model is acceptable for serverless cold-start systems that optimize restore and treat snapshot creation as offline work [20, 22, 24–29], but mismatched with agents, which checkpoint after nearly every action and restore on every backtrack or fork-out. Whole-VM snapshots pause and copy unrelated guest state (E2B reports ~ 4 s per GiB [17]), while process-level CRIU [16] avoids VM scope but still serializes and reloads process pages, file mappings, and descriptors.

Challenge-3: Coupled sandbox state requires explicit consistency management. Even if filesystem and process C/R are optimized in isolation, agents cannot restore the two dimensions independently. Process memory contains open descriptors, mmap’d files, Python import caches, tool outputs, and assumptions about packages or tests on disk; the filesystem records the edits and side effects that this memory will continue to use after rollback. A naive composition of overlays and CRIU can capture the two dimensions at different logical times, register a partially completed checkpoint after a failure, or resume a process before its target filesystem is installed. Agent sandboxes therefore need explicit consistency management for an atomic, failure-aware (filesystem, process) state pair, not two unrelated snapshots.

This paper observes that consecutive checkpoints in agent workloads differ only marginally, with only a few new files or modified memory pages between steps. Therefore, instead of duplicating the entire state, a sandbox should *only duplicate the changes between consecutive checkpoints* (**Key Insight**).

To this end, we present δ Sandbox, an OS-level rollbackable sandbox tailored for stateful agents. δ Sandbox achieves millisecond-level checkpoint/rollback through a new OS abstraction, δ State, which treats the filesystem and process memory as a transactional, change-based state pair. We introduce two co-designed OS mechanisms in δ Sandbox, coordinated by a StateManager:

Firstly, to support efficient filesystem C/R, we propose δ FS. δ FS extends overlays with *runtime layer switching*, which reconfigures the layer stack without unmounting or closing open file descriptors. This makes a checkpoint a metadata-only layer insertion that preserves history, while restore selects a recorded layer-stack configuration in $O(1)$ time; reference-counted generations keep cached dentries and inodes safe while old stacks drain. To keep the switch transparent to a live agent, δ FS adds a *lazy switch for open files*: per-inode version tags detect open files or mmap’d regions resolved before the checkpoint and re-resolve/copy them up on the next write. Finally, δ FS uses *XFS reflink* under overlays copy-up, so unchanged extents remain shared across

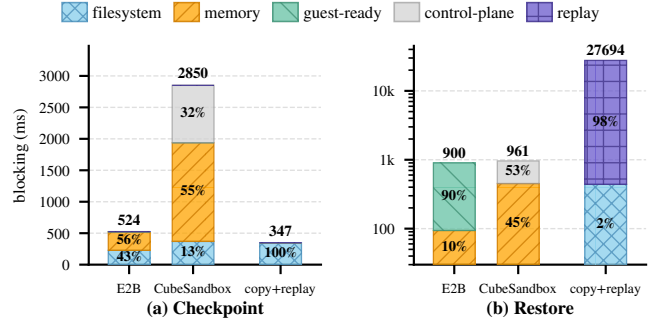


Figure 1. Per-event checkpoint/restore cost of existing approaches on SWE-Search, decomposed into filesystem, process/VM, and other phases (restore on a log axis).

renamed generations and file updates pay only block-level *copy-on-write* rather than file-size-proportional duplication.

Secondly, to C/R process memory state efficiently, we design δ CR. δ CR builds process checkpoints around a *warm-template* fast path: at a quiescent point, the agent inline-forks a frozen template while δ CR asynchronously dumps that template with CRIU, giving each checkpoint both a low-millisecond restore target and a durable fallback, with the work hidden inside the LLM I/O window. On a template hit, *fork-based restore* spawns the restored agent by copying page tables rather than memory pages; on a miss, *CRIU lazy-pages restore* reconstructs the process on demand and reinserts it into the pool. To keep deep search bounded, δ CR *skips read-only nodes* to avoid snapshots for cheap-to-recompute actions, and *reachability-aware garbage collection* reclaims only snapshots the search can no longer select. Finally, we propose *async-warm* to absorb post-fork CoW faults off the critical path. A *network proxy daemon* keeps LLM sockets and SDK threads outside the template so fork is safe; for RL fan-out, δ Sandbox implements batched `fork_n` on top of the same template protocol.

Thirdly, to keep memory and file views consistent, δ Sandbox adds a **StateManager** and a coupled checkpoint/restore protocol. The StateManager maintains a snapshot index tree mirroring the search tree, where each node records the target overlay layer-stack configuration, CRIU dump path, and live template when available. On checkpoint, a quiescent agent boundary lets δ CR capture the post-action process and δ FS freeze the matching filesystem generation before the node becomes visible; on restore, δ Sandbox switches the filesystem first and resumes the restored process only afterward. This ordering keeps memory and file views mutually consistent, and serves rollback, value-time test isolation, and batched RL fork-out.

We implement δ FS as an overlays kernel module and δ CR as a userspace framework, integrating both in δ Sandbox, an agent sandbox based on a Firecracker microVM. End-to-end evaluations show that a checkpoint’s 10.83 ms of work is hidden under inference (zero agent-perceived blocking),

while template-fork restore completes in a mean 1.86 ms; the CRIU lazy-pages fallback after template eviction restores in 9.29 ms. On SWE-bench MCTS workloads, δ Sandbox reduces state-management overhead from 23–48% of total time on the E2B baseline to 1–2%. For RL training fan-out, δ Sandbox’s batched fork_n creates up to 64 isolated rollout sandboxes from one warm template at 0.55–5.1 ms per child, over an order of magnitude faster than CubeSandbox and E2B.

2 Background and Motivation

2.1 Search and Exploration for AI Agents

Modern LLM-based agents increasingly rely on search and exploration, whether tree-structured or parallel sampling, to solve complex tasks such as math and code [1].

MCTS. MCTS is the tree-search paradigm, deciding through selection, expansion, evaluation, and backpropagation [32, 33]; LATS [6] adapts it to LLM agents, using UCT-guided selection to pick the most promising node and then generating and executing candidate actions in a sandbox. SWE-Search [34] specializes this loop to SWE-bench code repair, improving the success rate over single linear generation.

Best-of-N. BoN runs N independent trajectories from the same initial state and keeps the best by execution feedback [5, 10], i.e., N clones of the initial environment.

RL training. RL post-training fans out rollouts (i.e., inference and agent exploration) at every training step. Chain-based algorithms such as GRPO [12] launch N independent rollouts from the same initial state and score the completed trajectories; tree-based variants such as Tree-GRPO [35] instead branch additional rollouts from intermediate states of existing trajectories, sharing the common prefix instead of re-executing it. Either way the fan-out recurs at every training step, so sandbox cloning latency directly gates training throughput; tree-based RL additionally requires forking from arbitrary mid-trajectory states.

2.2 Agent Sandbox and Coupled Checkpoint/Restore

To support efficient state exploration, AI agents rely heavily on two critical primitives: *checkpoint* and *rollback*. Sandbox runtimes [18, 20] typically provide these primitives for isolated execution environments, allowing agents to execute code, capture milestone snapshots, and safely revert state after failures. RL fan-out (§2.1) uses the same mechanism: the runtime checkpoints one warm environment and restores it N times, once for each rollout.

While most modern agent frameworks utilize some form of checkpoint and rollback, their implementations are often inefficient or incomplete. E.g., frameworks like SWE-agent [36] and OpenHands [37] perform model reasoning on the host and cheaply reset the in-sandbox repository using `git worktree`. However, this approach fails to checkpoint the running process, leading to a loss of the execution context (e.g., debugging history). To preserve this context, current

frameworks interact with the environment via a persistent daemon process inside the sandbox (e.g., E2B’s `envd` [4]). In this setup, a rollback only undoes file edits, leaving the process state desynchronized. Recovering the true process state requires *replaying every prior action* from the initial state, which is a latency-heavy operation proportional to the search depth and may severely bottleneck the critical path of backtracking search.

Coupled C/R matters. The state of a sandbox spans two interdependent dimensions: the *filesystem* (FS) (working directories, installed packages) and the *process memory* (execution context, open file descriptors). A partial rollback inevitably desynchronizes them. If only the FS is rolled back, the still-live process retains memory of file contents that no longer exist on disk. Conversely, if only the process is rolled back, its restored memory operates on files that have been mutated beyond its expected state. Either mismatch breaks the strict consistency required for correct tree search, necessitating a truly coupled checkpoint/restore mechanism.

2.3 Why Existing Approaches Fall Short

We summarize existing agent sandboxes in Table.1.

Checkpoint latency matters in agents. Although sandboxing is well studied in contexts like serverless computing, existing optimizations predominantly focus on minimizing *restore* latency to mitigate cold starts [18, 20, 38, 39]. However, in agents, the *checkpoint* operation also sits squarely on the critical path: *an agent cannot proceed until its state is safely snapshotted*. This creates a new performance requirement that prior systems overlook.

Efficient coupled C/R is difficult. Furthermore, existing systems manage filesystem and process states in isolation, failing to provide the coupled C/R efficiency required by agents, as shown in Table.1. *Filesystem-only* methods (e.g., Git worktree [36]) version files but discard process memory, forcing expensive agent restarts. Conversely, *Process/VM-only* methods (e.g., Firecracker snapshots [18], CRIU) capture memory but fail to coordinate with the underlying block device, necessitating separate, uncoupled filesystem snapshots.

A quantitative analysis. To better understand the inefficiencies of existing approaches, we present a quantitative analysis comparing E2B [17], CubeSandbox [31], and a copytree+replay baseline (Table.1). As shown in Fig.1, E2B requires 524 ms for a checkpoint and 900 ms for a restore, measured locally on its self-hosted *incremental* snapshot path (Table.1’s ~ 4 s/GiB is its documented full-VM pause). In our measurements, its restore latency is dominated by waiting for the guest daemon (`envd`) to stabilize after VM resumption. CubeSandbox incurs an even higher checkpoint cost of over 2.8 s. The naive copytree+replay baseline performs worst, requiring nearly 28 seconds to restore state by completely re-executing the prior trajectory. These latencies are fundamentally caused by the uncoordinated, heavyweight

Table 1. Comparison of sandbox state management approaches for agents.

Approach	Ckpt Latency	Restore Latency	Write Amplif.	Mem. Sharing	FS State	Process State	Arbitrary Rollback	Low control overhead
Git worktree	100 ms–1 s	100 ms–1 s	$O(\text{files})$	✗	✓	✗	✓	✓
copytree	100 ms–10 s	100 ms–10 s	$O(\text{dir})$	✗	✓	✗	✓	✓
Docker commit + restart	50 ms–10 s	1–10 s	$O(\text{layer})$	✗	✓	✗	✓	✓
Btrfs/LVM snapshot	10–100 ms	10 ms–1 s	Low	✗	✓	✗	✓	✓
Firecracker VM snapshot	200 ms–2 s	120–700 ms	$O(\text{VM})$	✗	✗	✓	✓	✓
DSec [30]	—	WAL replay	$O(\log)$	✗	✗	✗	✗	✓
CubeSandbox [31]	2850.4 ms [§]	960.7 ms [§]	$O(\text{VM})$	✓	✓	✓	✓	✗
E2B [17]	~4 s/GiB	~1 s	$O(\text{VM})$	✗	✓	✓	✓	✗
δSandbox (ours)	10.83 ms	1.86 ms[¶]	$O(4\text{KB})$	✓	✓	✓	✓	✓

[§]CubeSandbox v0.3.0 (CubeCoW), measured locally per event on our SWE-bench MCTS workload (§6). [¶]Fast path: 1.86 ms. Slow path (CRIU lazy-pages fallback): 9.29 ms. Both are per-event means under the standard δ Sandbox configuration; see Table.3 for component breakdowns.

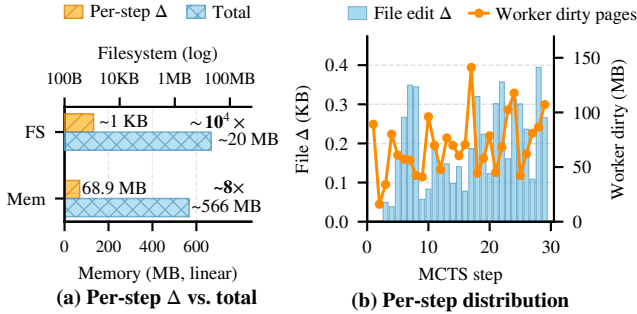


Figure 2. Each MCTS step changes only a tiny fraction of the total sandbox state. (a) Mean per-step change vs. total state for the filesystem and process memory. **(b)** Per-step filesystem edit size and worker dirty pages.

nature of existing C/R pipelines, which either rely on slow VM-granular snapshots or fail to tightly couple filesystem and memory state management. For workloads like RL fan-out, where these massive C/R costs are multiplied across every training step and branch point, such latencies create an insurmountable bottleneck, reinforcing the need for the millisecond-scale, coupled C/R primitives introduced by δ Sandbox.

2.4 Insight: Delta-Based Checkpoint and Restore

To understand the actual state mutation behavior of real-world agents, we analyzed the state changes during SWE-Search MCTS trajectories. As shown in Fig. 2, we observe that during each interaction step, an agent modifies only a minuscule fraction of the total sandbox state. Specifically, the per-step deltas of the process memory and the filesystem constitute on average only 12.2% and ~0.005% of the total state, respectively (Fig.2(a)). Furthermore, this highly localized mutation pattern remains consistent across different interaction rounds and workload scenarios.

This observation leads to our core idea: rather than duplicating the entire container state, an efficient agent sandbox should *only capture and restore the state deltas between consecutive checkpoints* (**Key Insight**). By focusing exclusively on the incremental changes, across both process memory and

filesystem dimensions, a sandbox can bypass the massive I/O and memory copy overheads of full-system C/R, enabling the millisecond-scale state management required for modern agentic search.

3 Overview

Based on the insight, we present δ Sandbox, a new agent sandbox realizing millisecond-scale C/R.

3.1 System Architecture

δ Sandbox extends existing sandboxes (e.g., microVMs) with two key components: δ FS and δ CR.

δ FS: Delta-based filesystem. A fundamental challenge for δ Sandbox is achieving highly efficient checkpoint and restore for filesystem states. A natural approach to this problem is applying copy-on-write (CoW) techniques, widely used in memory management (e.g., fork()), to the filesystem. By treating existing files as read-only and triggering copies only upon modification, a system could theoretically record just the state diffs between checkpoints.

Overlays [23] offers the closest realization of this idea. It provisions a writable upper layer for apps while maintaining a read-only lower layer that encapsulates the base sandbox image. However, overlays suffers from a critical limitation: its layer stack is *fixed and static* at mount time. Altering the layers during runtime requires a complete umount/mount cycle, which flushes caches and breaks active file descriptors, making it infeasible for running agent processes.

To resolve this, we introduce δ FS, a new file system extending overlayFS that supports dynamic, runtime overlay reconfiguration. When an agent completes a task step and requests a checkpoint, δ FS does not duplicate the underlying files. Instead, it dynamically freezes the current writable layer into a read-only state and seamlessly inserts a new writable upper layer on top of it. Consequently, all subsequent file updates trigger copy-on-write to this new upper layer. Through this mechanism, a checkpoint operation is reduced to a lightweight layer-insertion, and a rollback becomes a simple layer-removal, perfectly aligning with our diff-based C/R insight. δ FS further uses reflink-enabled XFS,

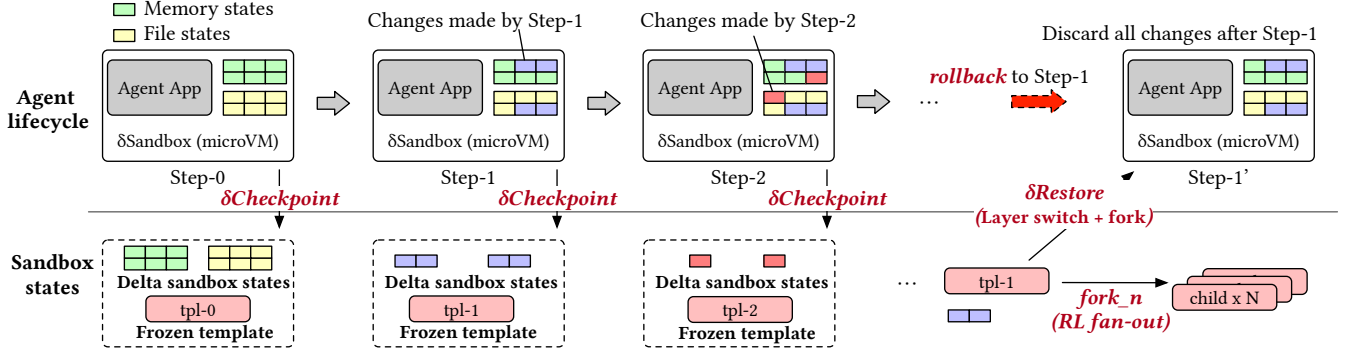


Figure 3. Design Overview. The agent worker runs inside the sandbox, so every step is C/R-protected. From one diff-based checkpoint/restore primitive, δ Sandbox serves two workloads: millisecond *rollback* to a prior node for tree search, and *N*-way *fork-out* from a frozen template for parallel fan-out.

whose block-level copy-on-write keeps copy-up cost proportional to the blocks actually written (Fig. 4.1).

δ CR: Delta-based process checkpoint/restore. Process-level state management has seen extensive prior research. Methods like CRIU [16] and `fork()` [20, 40] are widely used. However, applying them directly to agent workloads presents severe challenges. While recent efforts have optimized CRIU’s restore latency, its checkpoint latency, even using incremental dumps [41], remains 100–1,000 ms. While `fork()` offers milliseconds cloning, it introduces its own limitations: (1) it requires the parent process (template) to remain resident in memory, scaling poorly for agent exploration; (2) synchronous CoW page faults scatter latency across the agent’s critical path post-restore; and (3) both CRIU and `fork()` are only safe if the process holds no long-lived external state. Standard Python LLM SDKs (e.g., OpenAI) violate this safety requirement by spawning persistent HTTP/2 connection-pool threads in the background, which fatally deadlock upon a template fork.

To this end, δ Sandbox introduces δ CR, a hybrid process C/R mechanism that synthesizes the durability of CRIU with the speed of `fork()`. During a checkpoint, δ CR simultaneously executes an asynchronous CRIU incremental dump and a template-creating `fork()`, seamlessly hiding both operations within the LLM I/O window. During a restore, δ Sandbox prioritizes the fast-path by forking the resident template. To bound memory costs, δ CR employs a template pool, transparently falling back to CRIU’s lazy-pages restore if the template has been evicted. δ CR introduces an asynchronous pre-warming mechanism to eliminate runtime CoW faults from the critical path, alongside a Network Proxy Daemon that isolates persistent I/O connections outside the agent process, guaranteeing that the templated processes remain perfectly forkable.

Coordination and optimizations. To ensure atomic, coupled state transitions, δ Sandbox employs a two-tiered StateManager to orchestrate δ FS and δ CR, maintaining each checkpoint as one δ State: a consistent, transactional

(filesystem, memory) pair. The global Sandbox Controller runs on the host, maintaining a snapshot index tree isomorphic to the search tree, where each node records its CRIU dump path and overlay layer-stack configuration. Simultaneously, the Guest Daemon executes the local C/R sequences inside the VM, keeping state management off the host’s critical path (§5).

δ Sandbox also introduces agent-specific optimizations to further minimize overhead. First, a *semantic-aware lightweight skip* mechanism avoids unnecessary snapshots. Second, δ Sandbox utilizes a *reachability-aware garbage collector*, to reclaim unused checkpoints.

3.2 System Workflow

Fig. 3 shows the overall workflow of δ Sandbox. δ Sandbox efficiently supports two critical workloads: millisecond-scale rollback to a prior node for tree search, and rapid *N*-way fork-out from a frozen template for parallel RL training.

3.2.1 Coupled δ Checkpoint and δ Restore. We introduce the workflow of two primitives.

δ Checkpoint. A checkpoint operation is orchestrated by the StateManager and executed entirely off the host’s critical path by the Guest Daemon inside the sandbox. First, the daemon captures the process state via δ CR: the agent executes a `fork()` at a quiescent point to create a frozen template, while a CRIU incremental dump is asynchronously submitted. Concurrently, the daemon triggers the δ FS `ioctl` to freeze the current FS upper layer and install a fresh writable layer. The agent remains perfectly quiescent across both operations. This guarantees that every δ Checkpoint registers a strictly consistent (FS, memory) state delta. Because this entire sequence executes concurrently while the agent awaits its LLM response, the checkpoint cost is completely hidden behind inference latency.

δ Restore. A restore operation seamlessly reverses this process. When rollback, the Sandbox Controller instructs the Guest Daemon to discard the current active agent. The daemon then uses the δ FS `ioctl` to dynamically revert the FS

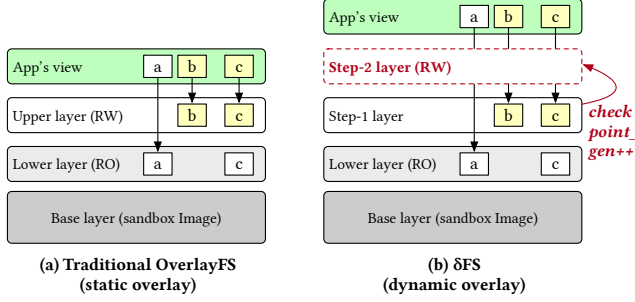


Figure 4. δ FS architecture. (a) Standard overlayfs has a static layer stack. (b) δ FS reconfigures it at runtime: a checkpoint inserts a fresh writable upper, and a rollback switches to the target configuration; `checkpoint_gen` stamps each switch with a new generation.

layer stack to the target checkpoint’s exact configuration. Simultaneously, δ CR rebuilds the process memory. If the target’s frozen template is still resident in the pool, δ CR executes a millisecond `fork()` (the fast path); otherwise, it falls back to a CRIU lazy-pages restore. The new agent process then resumes execution from the exact instruction immediately following the original checkpoint, entirely unaware of the rollback, with its FS and memory perfectly synchronized.

3.2.2 Fork-out. Fork-out composes the same two primitives differently: rather than restoring one past state, it instantiates N children from one checkpoint, chain-based RL from the warm initial state and tree-based RL from any mid-trajectory checkpoint (§2.1). The StateManager takes a δ Checkpoint at the source state, then forks the checkpoint’s frozen template N times in one batched `fork_n` request, each child receiving a private filesystem view over the frozen layers (§4.2.1). Each child starts with its memory CoW-shared with its siblings, at the same millisecond-scale per-child cost as a fast-path δ Restore (§6.2.2).

4 Detailed Design

4.1 δ FS: Dynamic Overlay Filesystem

Overlayfs fixes its layer stack at mount time. As shown in Fig. 4, δ FS releases this restriction and supports dynamic layers.

Interface. To expose this dynamic capability, δ FS introduces a `hotswitch` `ioctl`, allowing the controller to reconfigure the overlay layer stack in place. The `ioctl` accepts a `{mount, option, style}` configuration, which explicitly defines the target state: a writable upper layer, its associated workdir, and an ordered chain of read-only lower layers.

During a checkpoint, if the current upper is dirty, the StateManager uses this interface to sink it into a frozen lower layer and install a new writable upper layer above it. Consequently, files in the prior layer remain reachable, but any subsequent modifications trigger a copy-on-write to the new upper layer, successfully isolating the state delta. The restore

path is even simpler: the StateManager invokes the `ioctl` to specify the exact lower layer chain of the target checkpoint and installs a fresh writable upper layer above it. Because this process is purely a layer reconfiguration, it operates with $O(1)$ efficiency regardless of how far back in the search tree the target checkpoint lies. The `ioctl` is entirely synchronous; once it returns, subsequent path resolution observes the new stack; already-open descriptors are handled by the lazy rebinding mechanism below.

Layer lifetime. A hot-switch completes without freezing the filesystem, so open files and in-flight lookups may still reference the old layer stack after the `ioctl` returns. δ FS therefore reference-counts each retired stack: a cached overlay entry holds a reference to the stack it was resolved against, and the kernel frees a retired stack only after its last reference drops and an RCU grace period has elapsed.

Open files across checkpoint and restore. Files opened prior to a checkpoint present a challenge: their FDs may point to backing files within an outdated layer stack. δ FS resolves this via a generation-based lazy rebinding mechanism. Each inode caches the `checkpoint_gen` at which it was last resolved. During any file access, the kernel compares this cached generation against the current global filesystem generation. If the generations match, the access proceeds directly via the fast path with near-zero overhead. If they differ, the kernel re-resolves the inode against the new layer stack: read-side accesses only refresh stale references and keep reading from read-only layers, while write-side accesses first detach the descriptor from its stale backing and redirect it to the current writable upper, copying file data when needed. Both paths update the inode’s generation and backing under a per-inode mutex. Furthermore, if concurrent threads race to copy up the same file, the loser simply reuses the already-created upper layer rather than retaining its stale backing file.

Descriptor semantics. A rollback replaces the filesystem view underneath live file descriptors. δ FS makes this deterministic. After a checkpoint, writing through a stale descriptor copies up from its frozen backing into the current upper, so checkpointed history remains immutable. After a restore, a descriptor whose path exists in the target state rebinds reads and later copy-ups to that state. If the file existed only in a discarded upper, the descriptor continues through an anonymous backing copy and never resurrects the old path.

XFS reflink. δ FS can run over different backing FSES, but δ Sandbox uses reflink-enabled XFS to bound write amplification. For regular-file data copy-up, overlayfs first tries `vfs_clone_file_range`; on XFS this clones extents by reference, so the copied-up file shares physical blocks with its source until those blocks are overwritten. Reflink sharing composes across generations: an extent left unmodified across N checkpoints occupies one physical block.

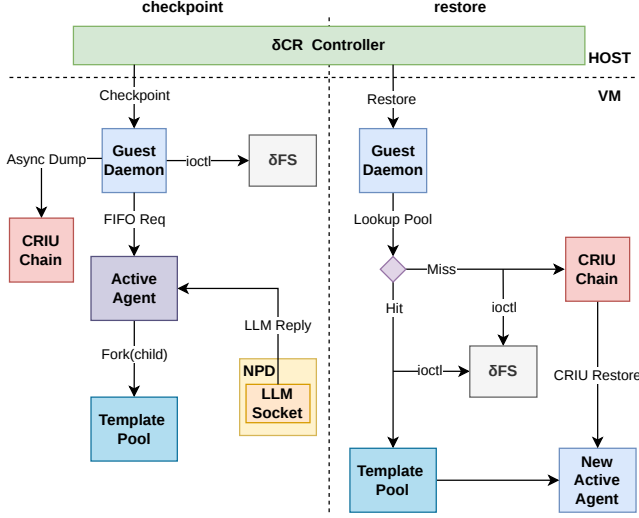


Figure 5. δCR architecture. Checkpointing creates both a CRIU image and a frozen template. Restores use the template fast path on hit, or the CRIU image on miss; the network proxy (NPD) keeps external I/O off the agent path.

4.2 δCR: Diff-based Checkpoint/Restore

δCR manages the process (memory) dimension of each checkpoint around two paths: a *warm-template* fast path, which checkpoints by forking off a frozen template and restores by forking it back in low milliseconds, and a durable CRIU [16] fallback, with incremental dumps via soft-dirty page tracking and lazy-pages restores via userfaultfd [42]. Fig. 5 summarizes its three components, the template pool (§4.2.1), the async-warm thread, and the network proxy (§4.2.3), together with the two restore paths.

4.2.1 Template Pool and Fork-Based Restore. The fast-path restore mechanism relies on a *template pool*, which maintains a registry of frozen (SIGSTOP’d) processes, each keyed to a specific snapshot ID.

Checkpoint flow. During checkpoint k , the Guest Daemon issues a fork request to the agent’s control FIFO; the agent forks at its next quiescent point (i.e., no syscalls in flight, no files half-written). The frozen child registers as *template_k* in the pool while the active worker continues in the main loop. A short-lived fork helper re-parents the template away from the active process. δCR then takes an incremental dump in the background as a durable fallback for crash recovery and cold-path restores.

Inference-masked checkpointing. Because a checkpoint runs while the agent awaits its LLM response, both the dump and the fork are hidden inside the inference window. The one obstacle is the agent’s live LLM connection; δSandbox offloads it to the network proxy (§4.2.3) so the request keeps progressing while the agent forks and the dump completes. The agent’s brief checkpoint pause (~11 ms in δSandbox) fits comfortably within the LLM inference latency.

Fast-path restore (template hit). To restore checkpoint k while *template_k* is alive, the Guest Daemon kills the current agent, reverts the layer stack, and forks *template_k*; the child resumes via SIGCONT from the instruction right after the original checkpoint, with the filesystem already switched to match. The fork duplicates only page tables, so restore completes in low milliseconds; an optional async-warm thread absorbs the post-fork CoW faults off the critical path. The C/R protocol makes the worker single-threaded at the moment it forks: the fork happens only at a quiescent point, after transient thread pools are shut down, and long-lived network threads live in the separate proxy process (§4.2.3). Inherently multi-threaded runtimes can be supported by adopting existing techniques from prior systems [20].

Slow-path restore (template miss). If *template_k* has been evicted, δCR falls back to CRIU lazy-pages restore. CRIU reconstructs the process skeleton, registers all anonymous VMAs with userfaultfd, and resumes the process, faulting in pages on demand from the dump images. To minimize critical-path latency, the background async-warm thread prefetches hot-zone pages off the critical path.

N -way fork-out (fork _{n}). RL fan-out (§3.2.2) reuses the template-fork protocol through a batching primitive that we implement in δSandbox: fork _{n} wakes the frozen template once, forks all N children in one cycle, and re-suspends it, eliminating $N-1$ per-child resume/suspend round-trips compared with issuing N one-at-a-time template restores. FS isolation is per child: the source’s overlay layers are frozen once at the branch point (one δFS ioctl), and each child receives its own overlay mount whose read-only lowers are the frozen chain and whose writable upper is private, so isolating a child costs one mount and no data copy. These mounts are pre-created before the fork, so every child resumes directly into a ready, private filesystem view.

4.2.2 Bounding Memory under Deep Search. By default δCR writes each CRIU dump to an image file on the FS, kept in the page cache for fast cold-path restore but reclaimable under memory pressure; a checkpoint’s *unreclaimable* resident cost is therefore just its frozen template (CoW page tables and privatised pages). Resident memory therefore tracks the live template pool, which grows as the MCTS tree deepens; left unbounded it would accumulate until the guest exhausts memory. δSandbox bounds it along three complementary axes: it *avoids* creating snapshots for cheap-to-recompute nodes, *caps* the resident template set, and *reclaims* snapshots the search can no longer reach.

Skipping read-only nodes. Many agent actions, such as grep, cat, and find, are read-only and idempotent: they leave the filesystem untouched and produce no agent state that a restore must reproduce. A semantic classifier inspects each action and, for such a node, skips both the CRIU dump and the layer switch, recording only a metadata marker (parent snapshot ID and action). A restore to that node simply

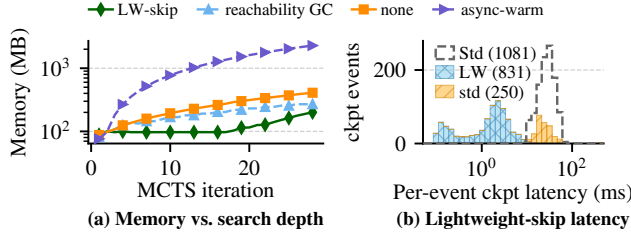


Figure 6. Bounding memory under deep search. (a) Resident memory (template pool and active worker) vs. MCTS iteration under four retention policies. (b) Per-event checkpoint latency with and without lightweight-skip.

re-issues the action on the parent’s state. Because read-only steps dominate a typical trajectory, this avoids creating most snapshots and sharply lowers the snapshot-store footprint, at the cost of a replay on restore. δ Sandbox therefore enables it only when snapshot memory, rather than restore latency, is the bottleneck.

Bounded template pool. When resident template memory becomes the bottleneck, the pool caps the live set at N_{tpl} frozen templates under LRU eviction. Evicting a template costs only latency but not correctness. Its CRIU image remains on the snapshot store, so a later re-selection falls back to the slow path, §4.2.1. The pool thus caps resident template memory regardless of how large the search tree grows.

Reachability-aware garbage collection. A recency- or visit-count policy is unsafe for MCTS: discarding a node UCT may still select, or an ancestor whose delta image a reachable node’s incremental dump chains back to, triggers a restore-fail re-selection loop that wastes the iteration budget. δ Sandbox instead drives GC from a persistent *node-to-image index*, in which each node records its parent, children, and delta-image ID. A pass marks the reachable set and reclaims the rest at leaf or whole-subtree granularity. A delta image is freed only when its node is unreachable *and* no reachable descendant or shared node depends on it, so reclamation never breaks the `-prev-images-dir` chain. Since an unreachable node is never re-selected, GC drops both its frozen template and, when on-disk image storage is scarce, its delta image. GC runs asynchronously, off the checkpoint critical path. Fig. 6a shows the combined effect: with no policy, resident memory climbs with tree depth; lightweight-skip and reachability-GC keep it flat. Lightweight-skip additionally diverts most read-only nodes to a near-free metadata write (Fig. 6b).

Async-warm. After a fork, the child shares all pages with the template copy-on-write, so its first write to each page takes a synchronous CoW fault. An async-warm thread in the Guest Daemon (keeping the agent single-threaded) absorbs these faults off the critical path: right after the fork it walks the child’s anonymous writable VMAs via `/proc/[pid]/mem`, forcing each page’s private copy in the background while the child runs. The latency win is not

free: privatising every page destroys CoW sharing with the template (over $5\times$ resident inflation, Fig. 6a) and marks every touched page soft-dirty, degrading the next incremental dump to a full one. δ Sandbox therefore disables async-warm by default, enabling it only when post-restore fault latency, rather than memory, is the bottleneck.

4.2.3 Reusable I/O during Checkpoint with a Network Proxy. Fork-based restore is safe only if the frozen template’s address space holds no long-lived external state, such as live network threads. However, standard LLM SDKs in Python (e.g., `openai` or `anthropic`) violate this requirement by spawning background HTTP/2 connection pools and `ThreadPoolExecutor` workers. Forking a template containing these threads leads to fatal issues: keep-alive TCP sockets survive as half-open file descriptors, connection-pool states diverge, and threads frozen mid-handshake permanently deadlock the frozen template.

δ Sandbox resolves this with a *network proxy daemon*, a separate process in the same sandbox that owns all LLM clients and connections. The agent never links the SDKs; it exchanges small FIFO tokens (each $\leq \text{PIPE_BUF}$, hence atomic) with the proxy over a shared directory, keeping every socket and SDK thread out of its address space. Being a separate process, the proxy is excluded from both the CRIU dump and the template fork, so a template carries only the agent’s own short-lived threads. During a checkpoint the agent pauses only to inline-fork; the in-flight LLM request keeps progressing inside the proxy, which buffers the response until the agent resumes. After a fork-based restore the child inherits the FIFO descriptors from the template, resuming communication with no reconnection protocol. δ Sandbox does not currently support network I/O rollback, which may incur external side effects.

4.3 StateManager Coupling Protocol

StateManager coordinates the system to enforce an invariant: *every saved state is a consistent (filesystem, memory) pair*.

Consistency protocol. During a checkpoint, consistency rests on a *quiescence invariant*: the worker replies only after finishing its action and returning to a single-threaded control point, where it blocks on its control FIFO until the next command. The fork (memory) and the δ FS ioctl (filesystem) therefore observe the same post-action state, and the asynchronous dump serializes the corresponding quiescent process state. During a restore, the restored child blocks at the same control point, and the StateManager issues it commands only after both the layer switch and the fork have completed, so the agent never executes against mismatched files.

Failure atomicity and concurrency. δ Sandbox is failure-atomic at the level of checkpoint visibility: a checkpoint becomes restore-visible only at its final registration step, so

a crash mid-sequence leaves the partial checkpoint unreferenced, and registered checkpoints keep their frozen layers, which are immutable. A slow-path restore that needs the CRIU image waits for any in-flight dump to complete; the fast path reads only the frozen template. On restore, the template fork proceeds concurrently with the layer switch (the two touch disjoint kernel state); the synchronization point is the next-command boundary above, not the fork itself.

Value-time test isolation. Evaluating a node often runs tests (e.g., SWE-Search [34]), whose side effects such as `__pycache__`, pytest caches, and temporary files would permanently pollute a stateful, in-process worker; stateless sandboxes avoid this only by discarding the sandbox. `δSandbox` instead uses its own millisecond C/R primitive as a transaction bracket: the StateManager checkpoints before the test, restores once the result is collected, and then records the result as the node’s observation and reward. This is only practical because `δSandbox` makes coupled C/R a millisecond-scale primitive rather than a second-scale sandbox operation.

5 Implementation and Deployment

`δSandbox` is deployed inside Firecracker [18] microVMs; the design is not Firecracker-specific and carries over to container- or other VM-based sandboxes. `δFS` adds ~ 1.8 K LOC of C to overlaysfs; it builds as a loadable kernel module against stock Linux 6.8. `δCR` is ~ 3.8 K LOC of Python orchestrating CRIU, the template pool, the async-warm thread, and the network proxy (§ 4.2). The fork-out path (§ 4.2.1) is a separate ~ 330 -LOC module that adds the batched `fork_n` op to the template pool’s control-FIFO protocol. CRIU dump images and overlay uppers both reside on the XFS rootfs, where reflink CoW is block-granular; recent dump images stay in the page cache for fast restore and are reclaimed under memory pressure.

Deployment model. The StateManager’s two tiers (§3) map onto a Host–Guest split: the host-side Sandbox Controller manages VM lifecycle and workload provisioning, while the in-VM Guest Daemon owns the entire per-event C/R sequence, keeping all latency-sensitive work in a closed loop inside the guest; the KVM host is unmodified. Because C/R is process-level, a single VM can host multiple independently checkpointable agents that share one kernel and the read-only base layers; rolling back one search branch leaves the others untouched, which a VM-level snapshot cannot do (it captures all processes at once).

Framework integration. `δSandbox` complements application-level frameworks such as LangGraph [43] and LangChain [44], whose checkpoints capture logical state only and cannot undo OS-level side effects. Integration can be transparent, pairing the framework’s state save/restore events (e.g., a LangGraph checkpoint saver) with coupled OS-level C/R, or explicit, exposing `sandbox_checkpoint/`

`sandbox_restore` tool nodes so the agent or search strategy controls physical rollback directly.

6 Evaluation

6.1 Experimental Setup

Hardware. All experiments run on a four-socket server (96 physical cores / 192 hardware threads), with experiment data and snapshot images placed on an NVMe SSD. RL training fan-out (Fig. 8b–c) additionally uses a single-node 4-GPU cluster (96 GB each), which also hosts the Qwen3-Coder-30B inference endpoint that provides the MCTS search trajectories. `δSandbox` (based on a Firecracker microVM) is configured with 4 vCPUs, 8 GB RAM, and an XFS rootfs block device plus a read-only XFS data image (target repository source and conda environment). Unless noted, experiments use `δSandbox`’s default configuration: each checkpoint takes an incremental CRIU dump plus a warm-template fork, while the lightweight-skip and async-warm optimizations (§ 4.2.2) are optional and left disabled.

Workloads. We draw four MCTS-trajectory archetypes from SWE-bench Verified (grouped in Table. 2): **Django** (fat process), **SymPy** (read-heavy exploration), **Scientific** (Astropy, Matplotlib, scikit-learn, Xarray; NumPy-heavy, process-dominated), and **Tools/small repos** (pylint, requests, pytest; lightweight).

Baselines. All baselines capture *both* dimensions; otherwise MCTS rollback determinism would break (§ 2.2). Throughout, `ck/rs` report each operation’s synchronous, request-to-return blocking on the agent’s critical path.

- **replay+cp:** filesystem state via `shutil.copytree` of the testbed directory; process state recovered by re-executing the recorded commands.
- **FC-Diff+dm:** filesystem state via a dm-snapshot CoW layer; process state via Firecracker’s incremental VM-memory snapshots.
- **CRIU+cp:** filesystem state via `shutil.copytree`; process state via a CRIU dump and restore of the agent process.
- **CubeSandbox** [31]: the self-hosted v0.3.0 runtime provides filesystem state via reflink snapshots and process state via soft-dirty incremental memory snapshots.
- **E2B(diff)** [4]: the self-hosted `e2b-dev/infra` runtime provides filesystem state via incremental rootfs snapshots and process state via incremental VM-memory snapshots. Note that Table. 2 and Fig. 7 measure this faster incremental path, not the documented full-VM pause in Table. 1 (~ 4 s/GiB).

6.2 End-to-End Performance

We evaluate two workloads that invoke coupled C/R at high frequency: MCTS search, where each tree expansion checkpoints at a parent node and restores at a new leaf; and RL rollout fan-out, where each training step forks N parallel rollout sandboxes from a single warm template.

6.2.1 MCTS Search Throughput. We evaluate MCTS performance in two stages. First, we replay recorded MCTS state-management events to measure per-event checkpoint and restore latency across the six backends in Table.2, which decomposes the results into checkpoint and restore phases. Second, we run 30-iteration trajectory replays on δ Sandbox and E2B to determine whether the per-event savings translate into lower end-to-end execution time (Fig. 7).

Checkpoint costs. δ Sandbox’s standard checkpoint creates a warm template and submits an asynchronous incremental CRIU dump that is masked by LLM inference (§4.2). The reported time is therefore the API call-to-return blocking interval and excludes dump completion. For `replay+cp`, checkpointing is a once-per-trace pristine-repository copy rather than a copy at every MCTS node. Firecracker Diff captures pages dirtied since the previous snapshot, taking 477–741 ms on average, while self-hosted E2B and CubeSandbox pay their VM-level snapshot costs.

Restore costs. Restore lies directly on the agent’s critical path because the next MCTS iteration cannot begin until it completes. δ Sandbox restores a checkpoint in 1.46–2.23 ms across workloads, with an event-weighted mean of 1.86 ms (Table.2). It achieves this low latency by overlapping the δ FS layer switch with a fork from the warm template; template-pool management and the slow-path fallback are discussed in §4.2.

The baselines perform substantially more work during restore. E2B resumes a VM at every event, taking 0.5–1.0 s. FC-Diff materializes a sparse copy of the base image and merges the diff chain along the MCTS ancestor path, taking 2.92–3.78 s. Replay cold-rebuilds the target search state, including agent startup, index loading, and action re-execution, taking approximately 27 s. E2B, FC-Diff, and replay are therefore approximately 480 $\times$, 1800 $\times$, and 15000 $\times$ slower than δ Sandbox, respectively.

We replay 30-iteration Qwen3-Coder-30B MCTS trajectories from the four workload groups on δ Sandbox and self-hosted E2B (diff), applying a checkpoint at every event for both systems. As shown in Fig. 7, δ Sandbox stays within 1.01–1.02 $\times$ of the LLM+action lower bound, whereas E2B reaches 1.30–1.93 $\times$; state management accounts for only 1–2% of total time on δ Sandbox versus 23–48% on E2B. This gap arises because δ Sandbox overlaps its incremental CRIU dump with LLM inference, leaving only millisecond-scale δ FS switching and restore on the iteration path, while E2B exposes its full checkpoint and restore latencies.

6.2.2 RL Rollout Fan-out. A motivating workload for δ Sandbox is RL training, in which a single warm template spawns N parallel children per training step. The underlying coupled fork-out primitive (§3.2.2) is cheap and scales well: across 9 SWE-bench MCTS trajectories, each per-child spawn (a template fork plus the δ FS overlay layer-switch) stays at 0.55–5.1 ms for N up to 64, and each child shares the

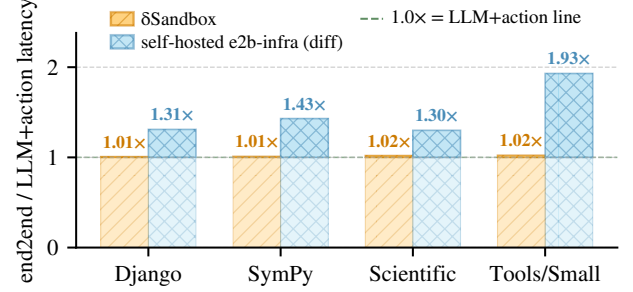


Figure 7. End-to-end time for 30-iteration MCTS trajectories, normalized per group to each system’s LLM+action latency (1.0 $\times$). To focus on the C/R substrate, we invoke E2B’s pause/resume directly and exclude its heavier control plane, which is orthogonal to the snapshot mechanism.

template’s pages copy-on-write so its marginal memory cost is only the pages it later writes; the fork-out primitive is thus not the fan-out bottleneck.

End-to-end sandbox fan-out. The preceding measurement isolates δ Sandbox’s batched fork-out primitive. We next evaluate whether this advantage carries over to complete sandbox fan-out, where each child must inherit the source process state and receive an isolated filesystem view. Fig. 8(a) compares the native fan-out paths of the three agent sandboxes. Starting from a frozen source that has touched 64 MiB of memory, each system creates N children using its native fork/clone interface (δ Sandbox `fork_n`, CubeSandbox `clone`, E2B `createSnapshot+create`) and is timed until every child has read back and verified the inherited state.

Across the measured range, δ Sandbox is an order of magnitude or more faster than both CubeSandbox and E2B (Fig. 8(a)).

Implication for synchronous RL training. Let T_{sandbox} denote the time to create all N rollout sandboxes (GPU idle), T_{gen} the GPU rollout-generation time, and T_{train} the GPU training time. Synchronous GPU occupation over a complete training step is then

$$U_{\text{sync}} = \frac{T_{\text{gen}} + T_{\text{train}}}{T_{\text{sandbox}} + T_{\text{gen}} + T_{\text{train}}}. \quad (1)$$

We measure T_{gen} using batched vLLM [45] and T_{train} using Qwen2.5-7B LoRA forward and backward passes across $N=1$ –64 rollouts (Fig. 8(b)). Applying Equation 1 at $N \in \{16, 64\}$ shows that δ Sandbox sustains near-saturating GPU occupation (95–97%), while CubeSandbox reaches 77–80% and E2B reaches 29–36% (Fig. 8(c)).

Asynchronous training. Production frameworks such as verl [46] (`fully_async`) and OpenRLHF [47] decouple the trainer from the rollout workers, removing sandbox fan-out from the training critical path at the cost of rollout *staleness*: a rollout is $S = (T_{\text{sandbox}} + T_{\text{gen}})/T_{\text{train}}$ policy versions old when consumed, reusing the same three quantities as Equation 1. We adopt $S < 1$ (at most one stale version) as the target. At

Table 2. Per-event mean blocking time (ms) on SWE-bench MCTS trajectories. *ck/rs* = checkpoint/restore; column naming follows *process-recovery+FS-recovery* (see §6.1 for baseline definitions).

Workload	replay+cp		FC-Diff+dm		CRIU+cp		CubeSandbox		E2B (diff)		δ Sandbox	
	ck [†]	rs	ck	rs	ck	rs	ck	rs	ck	rs	ck	rs
Django	568.1	28437	740.7	3784	791.7	1116	3297.9	800.1	487.7	790.2	12.12	2.23
SymPy	171.4	31591	476.9	3279	503.4	635.7	2732.6	1057.9	536.3	900.7	11.96	2.21
Scientific	265.9	28299	603.0	3256	435.7	655.1	2825.5	962.8	552.5	952.1	10.86	1.82
Tools/Small repos	86.4	20883	511.7	2920	361.4	427.1	2673.7	995.7	515.4	933.3	9.16	1.46
Event Avg	347.0	27694	622.3	3429	590.0	811.4	2850.4	960.7	524.4	899.7	10.83	1.86

[†]For replay+cp, ck is the per-trace pristine-repo copy and rs replays recorded completions with zero latency. Entries are per-event means; the Event Avg is event-weighted across archetypes by each backend’s own event counts.

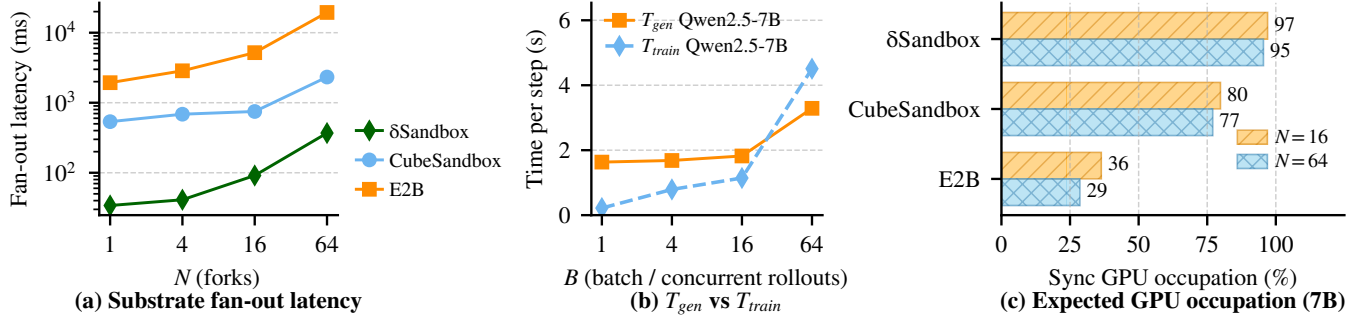


Figure 8. RL training fan-out characterization. (a) End-to-end time to fork $N \in \{1, 4, 16, 64\}$ memory-bearing children from one frozen source (64 MiB source, each child reads it back and verifies; E2B’s $N=64$ is 4 sequential 16-concurrency batches). (b) T_{gen} and T_{train} for Qwen2.5-7B (LoRA-r16, 4 GPUs). (c) Expected synchronous GPU occupation U_{sync} at $N \in \{16, 64\}$ (Equation 1).

$N=16$ the trainer step is short enough that all three backends exceed it: 1.67 for δ Sandbox, 2.25 for CubeSandbox, and 6.13 for E2B, with δ Sandbox closest. At $N=64$ the longer $T_{\text{train}}=4.51$ s pulls δ Sandbox below the target to 0.81, while CubeSandbox (1.25) and E2B (5.04) remain above it.

6.3 Extended Studies

6.3.1 Checkpoint/Restore Latency. We present the detailed latency breakdown in Table.3, which shows the cost of each component in δ Sandbox on SWE-bench.

Checkpoint latency. Per-step checkpoint latency is dominated by the template-fork tail; the incremental CRIU dump runs asynchronously (Table.3).

Restore latency. On the fast path, the template fork dominates, scaling linearly with agent RSS via page-table CoW, while the δ FS ioctl overlaps inside the fork window; the slow path is used only on the first restore or after GC eviction (Table.3).

6.3.2 Write Amplification. We measure per-edit *copy-up bytes* (file data re-materialized into the upper layer) and *physical I/O bytes* (loopback sectors written, including journal and metadata) over real SWE-Search agent edits sized 1–256 KB, across ext4, XFS, and XFS+reflink.

Copy-up savings. Reflink-aware copy-up shares unmodified extents with the lower layer, so only the 4 KB blocks an edit actually dirties are duplicated, independent of file size. Fig.9(a) shows ext4 and XFS-without-reflink coinciding at

Table 3. δ Sandbox per-component C/R latency (ms) over the SWE-bench MCTS replay (default incremental-dump path). The fast path forks the warm template (the common case); the slow path is the CRIU-lazy fallback on template eviction.

Component	ck	rs (fast)	rs (slow)
Overlay ioctl switch	0.07	0.19	0.25
Fork (stash / template)	8.87	1.34	—
CRIU C/R	async	—	8.72
Coordination [§]	—	0.52	0.57
Agent-perceived blocking	0	1.86	9.29

[§]Serialized coordination outside the overlapped phases. ^{||}Checkpoint’s 10.83 ms of local work overlaps the LLM-inference window, so agent-perceived ck blocking is 0 (§4.2).

every bin, both recopying the whole file on modification and growing linearly with size, while the reflink curve stays low and flat, widening the gap as files grow. Real SWE-Search edits touch only a small fraction of each file, so reflink duplicates a near-constant handful of blocks regardless of the file it lands in.

Physical I/O. Physical I/O reduction comes from both XFS metadata efficiency and reflink. Fig. 9(b) shows that XFS’s lighter metadata bookkeeping dominates on small files (132 KB $\rightarrow$ 26 KB at 1–8 KB, with reflink adding little), while reflink’s block sharing dominates on large files

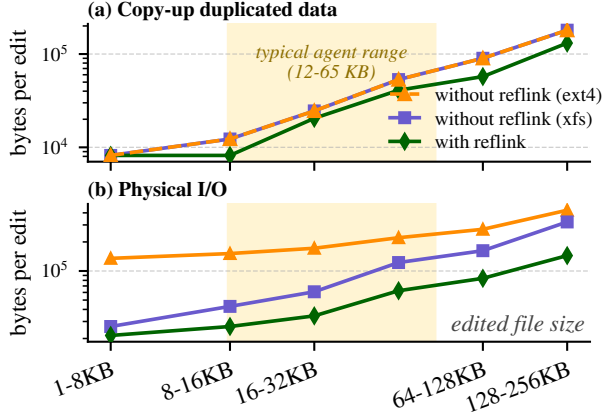


Figure 9. Per-edit copy-up bytes (a) and physical I/O bytes (b) vs. edited-file size (log-log) for real SWE-bench agent edits across ext4, XFS, and XFS+relink; per-bin medians. Shaded band: typical agent edit range.

(315 KB $\rightarrow$ 141 KB at 128–256 KB). The two mechanisms are complementary across the edit-size range.

6.3.3 Correctness. An integration suite drives the δ FS ioctl directly; its 53 checks cover checkpoint semantics across generations, open handles (stale and deleted-but-open descriptors, advisory locks), cache and mmap coherency, and data integrity over reflink, all passing with a clean kernel log. A separate normative test verifies the descriptor rules of Fig. 4.1 byte-for-byte. Malformed reconfigurations are rejected atomically with the mount still writable, and layer chains stay valid up to overlayfs’s 500-layer cap. Finally, the replays of §6.2 double as an end-to-end consistency check: each LLM request is hashed against the recording, and the hash covers all observations accumulated after each rollback, so any divergence between restored filesystem and memory breaks the replay; all replays completed with zero mismatches.

7 Related Work

Agent sandboxes. Recent agent sandboxes [4, 14, 15, 48] primarily optimize isolation and startup latency, but provide only coarse-grained rollback. Daytona’s OCI workspaces [14] use Docker-layer commits and do not capture live process state. E2B [4] uses Firecracker microVMs and pre-warmed templates, improving isolation but retaining VM-granularity snapshot costs. ZeroBoot [48] pushes VM cloning further with KVM-level CoW memory, but each branch remains an entire VM, with a shared block device, kernel-level dirty state, and no independent rollback for multiple in-VM trajectories. Yan’s fault-tolerant sandbox [15] moves toward transactional agent execution, but its filesystem rollback based on tool interception and userspace CoW simulation yields second-scale checkpoints and no coupled memory snapshot.

A position paper [49] frames agentic exploration as a systems problem and, like δ Sandbox, identifies fast forking and fork-aware state management as foundational gaps; it remains a diagnostic framing, whereas δ Sandbox contributes a concrete millisecond-scale OS mechanism.

Concurrent work Crab [50] tackles the same gap from a different angle. Crab proposes a semantics-aware approach using an eBPF-based inspector to determine checkpoint granularities. In contrast, δ Sandbox focuses on optimizing the full OS-level checkpoint/restore mechanism itself to achieve millisecond latency. The two approaches are complementary.

Sandboxes for serverless computing. Serverless systems have long optimized sandbox reuse, cold start, and snapshot restore, but their unit of reuse is a function invocation rather than a rollback point inside a long-lived agent trajectory. TrEnv-X [38] repurposes sandboxes across invocations using OS-level memory templates backed by CXL or RDMA memory pools, while earlier cold-start systems accelerate process creation, language runtime initialization, or post-restore fault handling for short-lived functions [19–21, 39, 51, 52]; Spice [52] moves restore-path fault resolution into the kernel with Overlay VMAs. They are orthogonal to δ Sandbox. A serverless instance typically restores a compute image and then runs one request, whereas δ Sandbox repeatedly descends and backtracks within one task.

Checkpointing optimizations. Existing checkpointing systems [53–58] provide important building blocks, but none provide the coupled, fine-grained rollback interface needed by stateful agents. CRIU [16] (which δ CR builds on) and DMTC [59] do process-level checkpoint/restore but not the coupled, high-frequency rollback agents require. Firecracker [18] and Sabre [60] restore whole VMs, too coarse for branches isolated inside one VM. Btrfs [61] and ZFS [62] snapshots give block-level CoW but need a separate storage stack, and EROFS [63] offers read-only overlay layers without a writable upperdir; δ FS instead hot-switches overlayfs layers over XFS reflinks. LangGraph [43] and LangChain [44] checkpoint only logical agent state, which cannot undo executed commands’ OS-level effects.

8 Conclusion

We present δ Sandbox, an OS-level rollbackable sandbox designed to accelerate agent workloads such as test-time tree search and reinforcement learning. Recognizing that subsequent agent states are highly similar, δ Sandbox eschews full state duplication in favor of diff-based checkpoint and restore. To achieve this, δ FS enables dynamic, unmount-free overlayfs layer switching for filesystem C/R, while δ CR utilizes CRIU dumps and warm-template forking for memory C/R.

References

- [1] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can

- Language Models Resolve Real-world Github Issues?. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acf6cbe9ca88fb84-Paper-Conference.pdf
- [2] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/file/4410c0711e9154a7a2d26f9b3816d1ef-Paper-Conference.pdf
 - [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
 - [4] E2B. 2026. E2B Documentation. <https://e2b.dev/docs>. Accessed: 2026-06-10.
 - [5] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 10131–10165. https://proceedings.iclr.cc/paper_files/paper/2025/file/1b623663fd9b874366f3ce019fd44-Paper-Conference.pdf
 - [6] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML ’24)*. JMLR.org, Article 2572, 23 pages.
 - [7] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 11809–11822. https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf
 - [8] Cheng Zhang, Erhu Feng, Xi Zhao, Yisheng Zhao, Wangbo Gong, Jiahui Sun, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. 2025. MobiAgent: A Systematic Framework for Customizable Mobile Agents. arXiv:2509.00531 [cs.MA] <https://arxiv.org/abs/2509.00531>
 - [9] The OpenClaw Project. 2026. openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way. <https://github.com/openclaw/openclaw>.
 - [10] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787 [cs.LG] <https://arxiv.org/abs/2407.21787>
 - [11] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2026. History Doesn’t Repeat Itself but Roll-outs Rhyme: Accelerating Reinforcement Learning with RhymeRL. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS ’26)*. Association for Computing Machinery, New York, NY, USA, 929–945. <https://doi.org/10.1145/3779212.3790172>
 - [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>
 - [13] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, juncai liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc., 113222–113244. https://proceedings.neurips.cc/paper_files/paper/2025/file/a4277440d50f1f15d2cb4c14f7e0cd2-Paper-Conference.pdf
 - [14] Daytona. 2024. Daytona. <https://daytona.io>.
 - [15] Boyang Yan. 2025. Fault-Tolerant Sandboxing for AI Coding Agents: A Transactional Approach to Safe Autonomous Execution. arXiv:2512.12806 [cs.AI] <https://arxiv.org/abs/2512.12806>
 - [16] The CRIU Project. 2011. CRIU: Checkpoint/Restore In Userspace. <https://criu.org>.
 - [17] E2B. 2026. E2B Sandbox persistence. <https://e2b.dev/docs/sandbox/persistence>.
 - [18] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: lightweight virtualization for serverless applications. In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation (Santa Clara, CA, USA) (NSDI’20)*. USENIX Association, USA, 419–434.
 - [19] Lixiang Ao, George Porter, and Geoffrey M. Voelker. 2022. FaaSnap: FaaS made fast using snapshot-based VMs. In *Proceedings of the Seventeenth European Conference on Computer Systems (Rennes, France) (EuroSys ’22)*. Association for Computing Machinery, New York, NY, USA, 730–746. <https://doi.org/10.1145/3492321.3524270>
 - [20] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS ’20)*. Association for Computing Machinery, New York, NY, USA, 467–481. <https://doi.org/10.1145/3373376.3378512>
 - [21] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS ’21)*. Association for Computing Machinery, New York, NY, USA, 559–572. <https://doi.org/10.1145/3445814.3446714>
 - [22] Xiaohu Chai, Tianyu Zhou, Keyang Hu, Jianfeng Tan, Tiwei Bie, Anqi Shen, Dawei Shen, Qi Xing, Shun Song, Tongkai Yang, Le Gao, Feng Yu, Zhengyu He, Dong Du, Yubin Xia, Kang Chen, and Yu Chen. 2025. Fork in the road: reflections and optimizations for cold start latency in production serverless systems. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation (Boston, MA, USA) (OSDI ’25)*. USENIX Association, USA, Article 28, 20 pages.
 - [23] Neil Brown. 2026. Overlay Filesystem. <https://docs.kernel.org/filesystems/overlayfs.html>.
 - [24] Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, and Ana Klimovic. 2024. Dirigent: Lightweight Serverless Orchestration. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (Austin, TX, USA) (SOSP ’24)*. Association for Computing Machinery, New York, NY, USA, 369–384. <https://doi.org/10.1145/3694715.3695966>
 - [25] Dong Du, Qingyuan Liu, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2022. Serverless computing on heterogeneous computers. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS ’22)*. Association for Computing Machinery,

- New York, NY, USA, 797–813. <https://doi.org/10.1145/3503222.3507732>
- [26] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. 2022. RunD: A Lightweight Secure Container Runtime for High-density Deployment and High-concurrency Startup in Serverless Computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 53–68. <https://www.usenix.org/conference/atc22/presentation/li-zijun-rund>
- [27] Hanfei Yu, Rohan Basu Roy, Christian Fontenot, Devesh Tiwari, Jian Li, Hong Zhang, Hao Wang, and Seung-Jong Park. 2024. Rainbow-Cake: Mitigating Cold-starts in Serverless with Layer-wise Container Caching and Sharing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 335–350. <https://doi.org/10.1145/3617232.3624871>
- [28] Jialiang Huang, MingXing Zhang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Yongwei Wu. 2024. TrEnv: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 421–437. <https://doi.org/10.1145/3694715.3695967>
- [29] Ariel Szekely, Adam Belay, Robert Morris, and M. Frans Kaashoek. 2024. Unifying serverless and microservice workloads with SigmaOS. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 385–402. <https://doi.org/10.1145/3694715.3695947>
- [30] DeepSeek-AI. 2026. *DeepSeek-V4 Technical Report*. Technical Report. DeepSeek-AI. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf
- [31] Tencent Cloud. 2026. CubeSandbox v0.3.0: Instant, Concurrent, Secure & Lightweight Sandbox for AI Agents. <https://github.com/TencentCloud/CubeSandbox/releases/tag/v0.3.0>. CubeCoW Copy-on-Write snapshot engine (relink volume + soft-dirty memory); accessed June 2026.
- [32] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML)*. 282–293.
- [33] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 1–43.
- [34] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2025. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 64485–64515. https://proceedings.iclr.cc/paper_files/paper/2025/file/a1e6783e4d739196cad3336f12d402bf-Paper-Conference.pdf
- [35] Yuxiang Ji, Ziyu Ma, Yong Wang, Guanhua Chen, Xiangxiang Chu, and Liaoni Wu. 2026. Tree Search for LLM Agent Reinforcement Learning. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=ZpQwAFhU13>
- [36] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 50528–50652. <https://doi.org/10.52202/079017-1601>
- [37] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 65882–65919. https://proceedings.iclr.cc/paper_files/paper/2025/file/a4b6ad6b48850c0c331d1259fc66a69c-Paper-Conference.pdf
- [38] Jialiang Huang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Yongwei Wu, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Mingxing Zhang. 2026. TrEnv-X: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. *ACM Transactions on Computer Systems* (March 2026). <https://doi.org/10.1145/3805475>
- [39] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference (ATC)*. USENIX Association, 57–70. <https://www.usenix.org/conference/atc18/presentation/oakes>
- [40] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhang Yang, Rong Chen, and Haibo Chen. 2023. No Provisioned Concurrency: Fast RDMA-codedigned Remote Fork for Serverless Computing. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 497–517. <https://www.usenix.org/conference/osdi23/presentation/wei-rdma>
- [41] The CRIU Project. 2016. Incremental dumps. https://criu.org/Incremental_dumps.
- [42] The CRIU Project. 2019. Userfaultfd. <https://criu.org/Userfaultfd>.
- [43] LangChain, Inc. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. <https://github.com/langchain-ai/langgraph>.
- [44] LangChain, Inc. 2022. LangChain: Building Applications with LLMs through Composability. <https://github.com/langchain-ai/langchain>.
- [45] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP '23)*.
- [46] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*.
- [47] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. 2024. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143 [cs.AI] <https://arxiv.org/abs/2405.11143>
- [48] ZeroBoot. 2026. ZeroBoot: Sub-millisecond VM Sandboxes for AI Agents via Copy-on-Write Forking. <https://github.com/zerobootdev/zeroboot>.
- [49] Jiakai Xu, Tianle Zhou, Eugene Wu, and Kostis Kaffes. 2025. Toward Systems Foundations for Agentic Exploration. arXiv:2510.05556 [cs.DC] <https://arxiv.org/abs/2510.05556>
- [50] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. 2026. Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes. arXiv:2604.28138 [cs.OS] <https://arxiv.org/abs/2604.28138>
- [51] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems* (Heraklion, Greece) (EuroSys '20). Association for Computing Machinery, New York, NY, USA, Article 32, 15 pages. <https://doi.org/10.1145/3342195.3392698>

- [52] Ben Holmes, Baltasar Dinis, Lana Honcharuk, Joshua Fried, and Adam Belay. 2025. Taming Serverless Cold Starts Through OS Co-Design. arXiv:2509.14292 [cs.OS] <https://arxiv.org/abs/2509.14292>
- [53] Yanning Yang, Dong Du, Haitao Song, and Yubin Xia. 2024. On-demand and Parallel Checkpoint/Restore for GPU Applications. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (Redmond, WA, USA) (SoCC '24). Association for Computing Machinery, New York, NY, USA, 415–433. <https://doi.org/10.1145/3698038.3698510>
- [54] P. Tullmann, J. Lepreau, B. Ford, and M. Hibler. 1996. User-level checkpointing through exportable kernel state. In *Proceedings of the Fifth International Workshop on Object-Oriented in Operation Systems*. 85–88. <https://doi.org/10.1109/IWOOS.1996.557874>
- [55] Dirk Vogt, Armando Miraglia, Georgios Portokalidis, Herbert Bos, Andy Tanenbaum, and Cristiano Giuffrida. 2015. Speculative Memory Checkpointing. In *Proceedings of the 16th Annual Middleware Conference* (Vancouver, BC, Canada) (Middleware '15). Association for Computing Machinery, New York, NY, USA, 197–209. <https://doi.org/10.1145/2814576.2814802>
- [56] A. Dearle and D. Hulse. 1995. On page-based optimistic process checkpointing. In *Proceedings of International Workshop on Object Orientation in Operating Systems*. 24–32. <https://doi.org/10.1109/IWOOS.1995.470583>
- [57] Emil Tsalapatis, Ryan Hancock, Tavian Barnes, and Ali José Mashizadeh. 2021. The Aurora Single Level Store Operating System. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (SOSP '21). Association for Computing Machinery, New York, NY, USA, 788–803. <https://doi.org/10.1145/3477132.3483563>
- [58] James S. Plank, Micah Beck, Gerry Kingsley, and Kai Li. 1995. Libckpt: transparent checkpointing under Unix. In *Proceedings of the USENIX 1995 Technical Conference Proceedings* (New Orleans, Louisiana) (TCON'95). USENIX Association, USA, 18.
- [59] Jason Ansel, Kapil Arya, and Gene Cooperman. 2009. DMTCP: Transparent checkpointing for cluster computations and the desktop. In *Proceedings of the 2009 IEEE International Symposium on Parallel and Distributed Processing (IPDPS '09)*. IEEE Computer Society, USA, 1–12. <https://doi.org/10.1109/IPDPS.2009.5161063>
- [60] Nikita Lazarev, Varun Gohil, James Tsai, Andy Anderson, Bhushan Chitlur, Zhiru Zhang, and Christina Delimitrou. 2024. Sabre: hardware-accelerated snapshot compression for serverless MicroVMs. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 1, 18 pages.
- [61] The Btrfs Project. 2009. Btrfs Documentation. <https://btrfs.readthedocs.io>.
- [62] OpenZFS. 2013. OpenZFS Documentation. <https://openzfs.github.io/openzfs-docs/>.
- [63] Linux Kernel Project. 2018. EROFS: Enhanced Read-Only File System. <https://erofs.docs.kernel.org>.